

```

;clc

;clear

;close all

=====
Faulhaber Polynomial Coefficients %
Using Vandermonde Matrix %
=====

;Kmax = 20

-----%
Matrix to store coefficients %
Row k contains coefficients of power k %
-----%

;CoeffMatrix = sym(zeros(Kmax,Kmax+1))

for k = 1:Kmax

;fprintf('Computing k = %d\n',k)

;N = k + 1

-----%
Construct Vandermonde Matrix %
-----%

;A = sym(zeros(N,N))

```

```

for m = 1:N
    for j = 1:N
        ;A(m,j) = sym(m)^(N-j+1)
    end
end

-----%
Construct RHS %
-----%
;b = sym(zeros(N,1))

for m = 1:N

    ;s = sym(0)

    for r = 1:m
        ;s = s + sym(r)^k
    end

    ;b(m) = s

end

-----%
Solve the system %
-----%
;coeff = A\b

```

```

-----%
Store coefficients in the matrix %
-----%

;'.CoeffMatrix(k,1:N) = coeff

-----%
Display coefficients %
-----%

;fprintf('\n')
;fprintf('=====\n')
;fprintf('Power k = %d\n',k)
;fprintf('=====\n')

for i = 1:N
;fprintf('%s\n',char(coeff(i)))
end

end

=====
Display complete matrix %
=====

(' ')disp
('=====' )disp
disp('Matrix of Faulhaber Coefficients')
('=====' )disp

```

```

disp(CoeffMatrix)

=====%%
=

Version 1 : Column Analyzer %%

=====%%
=

(' ')disp
('=====')disp
disp('COLUMN ANALYSIS')
('=====')disp

syms k

;[numRows,numCols] = size(CoeffMatrix)

for col = 1:numCols

-----%
Ignore zero columns %
-----%

;idx = find(CoeffMatrix(:,col)~=0)

if length(idx)<2
continue
end

;x = idx
;y = CoeffMatrix(idx,col)

```

```

-----%
Polynomial interpolation (Lagrange) %
-----%

;P = sym(0)

;n = length(x)

for i=2:n

;L = sym(1)

for j=2:n

if i~=j
;L = L*(k-x(j))/(x(i)-x(j))
end

end

;P = expand(P+y(i)*L)

end

;P = simplify(P)

-----%
Degree %

```

-----%

;d = feval(symengine,'degree',P,k)

-----%

Factorization %

-----%

;F = factor(P)

-----%

Roots %

-----%

;R = solve(P==0,k)

-----%

Leading coefficient %

-----%

;a = coeffs(P,k,'All')

;LeadCoeff = a(1)

-----%

Display %

-----%

;fprintf('\n')

;fprintf('=====\n')

;fprintf('Column %d\n',col)

;fprintf('=====\n')

disp('Polynomial:')

```
disp(P)
```

```
disp('Degree:')
```

```
disp(d)
```

```
disp('Factorization:')
```

```
disp(F)
```

```
disp('Roots:')
```

```
disp(R)
```

```
disp('Leading coefficient:')
```

```
disp(LeadCoeff)
```

```
end
```